

EE-5102/CS-6304 - Advanced Topics in Machine Learning – Fall 2026

Programming Assignment 1 – Release Date: 11 September 2026

Instructions

- This is the first graded programming assignment of the course.
- **Individual submission is mandatory.** Your code, experiments, results, analysis, and report must be your own. You may not collaborate with other students or share assignment work.
- The assignment is due on **25 September 2026 via LMS**.
- All submitted code must be available in a public GitHub repository.
- Submit an **8-page NeurIPS-style PDF report** presenting your experiments, results, and analysis. References do not count toward the page limit. Include the link to your GitHub repository at the end of the abstract. Further report-writing and submission requirements are provided in the Report and Submission Guidelines section.
- **AI Usage Policy:** You may not use generative AI to write any part of your PDF report. The report’s language, interpretation, and analysis must be entirely your own.
- **Coding Assistance:** You may use public libraries, built-in functions, open-source implementations, and coding assistance from an LLM. However, you are responsible for every submitted line of code and must understand how it works. Clearly attribute materially reused external code in your README.
- You are highly encouraged to read through each task before starting it.
- Fix random seeds where practical, record all reported hyperparameters, and preserve the configurations used to produce your results. Extensive hyperparameter tuning is not expected unless a task explicitly requires it.

Overview & Motivation

Standard supervised learning generally assumes that training and test data are drawn from the same distribution and share the same label space. These assumptions are frequently violated in deployment. An object from a known class may appear with a different texture, color, position, or style; test data may come from a new domain; or an input may belong to a class that was not observed during training. Consequently, a model can achieve strong in-distribution accuracy while failing under deployment conditions because it relies on cues or statistical regularities that do not remain stable.

This assignment examines four aspects of learning beyond the Independent and Identically Distributed (IID), closed-set setting.

Task 1: Inductive Biases and Representations. You will use controlled interventions to study how ResNets, Vision Transformers, and CLIP depend on shape, texture, color, and spatial structure. You will also examine whether the information encoded in a model’s representation agrees with the evidence that controls its final prediction.

Task 2: Domain Adaptation. You will investigate whether access to unlabeled target-domain examples can improve recognition by aligning source and target representations. You will also test whether stronger alignment can remove class-discriminative information or cause negative transfer.

Task 3: Domain Generalization. You will remove access to the target domain during training and model selection. You will compare ordinary empirical risk minimization with source-domain alignment and sharpness-aware optimization, asking whether domain invariance or local parameter stability improves generalization to an unseen domain.

Task 4: Open-Set Recognition. You will examine how a classifier can identify inputs from unknown classes rather than confidently assigning every input to one of its known classes. You will compare both post-hoc novelty scores and methods that change the learned representation or decision boundary.

Across all four tasks, the objective is to connect observed performance to the representation, architecture, and training objective that produced it. Lower domain discrepancy does not necessarily imply better class separation; higher closed-set accuracy does not necessarily solve open-set recognition; and models with similar clean accuracy may rely on substantially different evidence.

Use controlled comparisons, report aggregate results alongside informative failure cases, and explain why the observed behavior is plausible. Public implementations may be used where appropriate, but you must understand each method’s objective, permitted information, and training procedure well enough to implement, analyze, and debug it.

1 Inductive Biases and Feature Representations

Introduction and Terminology

Models with similar clean accuracy may rely on different evidence. In this task, you will compare a convolutional network, a Vision Transformer, and CLIP under controlled image interventions. The goal is to determine which visual cues influence their predictions, which cues remain encoded in their representations, and how these behaviors relate to architecture and pretraining.

An **inductive bias** is a preference introduced by a model’s architecture, data, or training objective. A **feature representation** or **embedding** is the vector produced before the final classifier. A **frozen backbone** is a pretrained feature extractor whose parameters are not updated; a **classifier head** is the linear classifier trained on top of those fixed features. A **controlled intervention** changes one visual factor while preserving others as far as possible. **Prediction consistency** is the fraction of images whose predicted class remains unchanged after an intervention.

Readings

- Geirhos et al. (2019) – *ImageNet-trained CNNs are biased towards texture; increasing shape bias improves accuracy and robustness*. Focus on cue-conflict evaluation and the definition of shape bias.
- Optional: Raghu et al. (2021) – *Do Vision Transformers See Like Convolutional Neural Networks?* Useful for comparing CNN and ViT representations.
- Optional: Huang and Belongie (2017) – *Arbitrary Style Transfer in Real-time with Adaptive Instance Normalization*. A practical basis for generating cue conflicts.

Dataset, Models, and Experimental Setup

Choose one of the following datasets:

- **STL-10** (recommended): ten distinct object classes and lower computational cost.
- **Oxford-IIIT Pets**: higher-resolution images and a more fine-grained classification problem.

Use the following pretrained backbones:

- torchvision ResNet-50 with `ResNet50_Weights.IMAGENET1K_V2`;
- torchvision ViT-B/16 with `ViT_B_16_Weights.IMAGENET1K_V1`;
- OpenCLIP ViT-B-32 with `pretrained='openai'`.

Freeze every backbone and extract its final representation: the global-average-pooled ResNet feature, the final ViT class token, and the normalized CLIP image embedding. Create a stratified 80/20 training/validation split from the official training partition using seed 6304. Train one linear classifier head for each backbone for at most 50 epochs using AdamW with learning rate 10^{-3} , weight decay 10^{-4} , and early stopping after five epochs without improved validation accuracy. Use seed 6304 for the classifier-head comparison. Also evaluate CLIP zero-shot using the fixed prompt “a photo of a {class}.” Fine-tuning and prompt search are outside the required comparison.

Select a class-balanced subset of 500 official test images using seed 6304. If a class has insufficient examples, use all available examples and document the imbalance. Save the selected image identifiers. Every model must receive the same clean and transformed images. Construct interventions on a common 224×224 RGB image before applying each model’s required normalization.

Your experimental-design choices are the dataset, cue-conflict class pairs and style strength, one additional color intervention, and the representation-visualization method and settings. State a hypothesis and an appropriate metric for each experimental choice before interpreting its result.

Steps

Complete the following experiments in order. The clean evaluation establishes a common reference; the subsequent interventions isolate particular visual cues; and the final analysis asks whether changes in predictions are also visible in the underlying representations.

1. Clean Baseline

Begin by evaluating the three classifier heads and zero-shot CLIP on the clean evaluation subset. Report top-1 accuracy, macro-F1, and mean maximum confidence; for zero-shot CLIP, compute confidence from the softmax over scaled class similarities. These results provide the reference for every later intervention and show whether the models begin with substantially different classifier quality. Consequently, compare interventions using both absolute performance and the change relative to each model’s own clean baseline.

2. Color Bias

Next, alter color while preserving object geometry to examine whether chromatic information contributes to the models’ decisions. Evaluate grayscale as the common intervention, then choose one additional transformation: fixed hue rotation, palette transfer, or class-swapped color statistics. Grayscale tests the effect of removing color, whereas the chosen transformation should test sensitivity to changing it. Explain what your transformation changes and preserves, and report the resulting accuracy change and prediction consistency relative to the corresponding clean images.

3. Shape vs. Texture

On ordinary images, shape and texture usually support the same label, so a correct prediction does not reveal which cue the model preferred. Construct cue-conflict images in which the content supplies the shape of one class and the style supplies the texture of another. Use AdaIN or another public style-transfer implementation, select at least five unordered class pairs, and generate both directions when feasible:

$$\text{shape/content} = A, \text{ texture/style} = B, \quad \text{shape/content} = B, \text{ texture/style} = A.$$

Produce at least 200 valid conflicts, balanced across class pairs and directions as closely as possible. Because failed stylizations would make the comparison ambiguous, define a visual rejection rule before model evaluation and record the accepted and rejected counts. Do not use model predictions to decide which images to retain.

Classify each prediction as the content/shape label, the style/texture label, or another label. This distinction matters because a shape-bias score ignores predictions belonging to neither intended class. Report both

$$\text{Shape Bias}(\%) = \frac{N_{\text{shape}}}{N_{\text{shape}} + N_{\text{texture}}} \times 100, \quad \text{Coverage}(\%) = \frac{N_{\text{shape}} + N_{\text{texture}}}{N_{\text{total}}} \times 100.$$



Figure 1: Illustrative stylization for shape-texture analysis. Your cue conflicts must be generated from the selected Task 1 dataset and retain interpretable content and style cues.

4. Translation

Test whether predictions remain stable as the object is displaced from its original location. Although convolution encourages translation equivariance, stride, pooling, padding, positional encoding, and training data can all make the final classifier position-sensitive. Translate each image by 0, 8, 16, and 32 pixels in the four cardinal directions

using reflection padding followed by a shifted crop, and average the results across directions. Plot accuracy and prediction consistency against displacement, defining consistency as

$$\text{Consistency}(\delta) = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{y}(x_i) = \hat{y}(T_\delta(x_i))]$$

5. Patch Structure

Now disrupt global spatial organization while retaining the image’s pixels and much of its local evidence. Divide each image into a 4×4 pixel-space grid and generate one non-identity patch permutation per image using seed 6304. Reuse exactly the same shuffled images across models and report the accuracy drop and prediction consistency relative to clean inputs. These intervention patches do not need to match the ViT token size: the experiment concerns reliance on coherent spatial arrangement rather than the mechanics of a particular architecture.

6. Representation Analysis

Finally, examine whether the preceding interventions alter the representation even when the predicted class remains stable. For grayscale, cue conflict, translation, and patch shuffling, pair each transformed image with its clean counterpart and measure the cosine stability of each backbone’s representation:

$$I_T = \frac{1}{N} \sum_{i=1}^N \frac{f(x_i)^\top f(T(x_i))}{\|f(x_i)\|_2 \|f(T(x_i))\|_2}.$$

Visualize the clean and transformed representations using either t-SNE or UMAP. For each backbone, fit one two-dimensional projection to the combined clean and transformed features so that both conditions appear in the same space. Use color to identify the ground-truth class and marker style to distinguish clean from transformed examples. t-SNE emphasizes which examples remain local neighbors, while UMAP also aims to preserve broader neighborhood structure; neither method preserves every distance from the original feature space. Keep the image subset and random seed fixed, report the visualization settings, and do not compare absolute coordinates across projections fitted separately for different backbones. Use the plots to discuss class separation, mixing between conditions, and transformations that visibly move examples away from their clean clusters.

What to Watch For

- Do not assume in advance that a CNN must be texture-biased or that a ViT must be shape-biased. Architecture, pretraining data, augmentation, model capacity, and positional encoding are confounded.
- Shape bias must be interpreted with coverage. A high shape-bias value based on very few shape-or-texture decisions is weak evidence.
- A confident prediction after patch shuffling is not necessarily a sensible prediction. Examine what evidence remains after global structure is destroyed.
- Prediction stability and representation stability answer different questions. A cue may remain encoded even when the classifier does not use it, or a prediction may remain unchanged despite a large representation shift.

Required Evidence

- A compact comparison of clean, grayscale, additional-color, and patch-shuffle performance.
- Shape, texture, and other decision counts, together with shape bias and coverage.
- A translation curve showing accuracy or consistency against displacement.
- Representation-stability results for all required interventions and a t-SNE or UMAP visualization comparing clean and transformed features.
- A small set of informative cue-conflict agreements, disagreements, or failures with model predictions.

Research Questions

Address the following in your report:

1. What do the color and cue-conflict experiments jointly reveal about each model’s reliance on shape, texture, and color? How does coverage affect the strength of your conclusions about shape bias?
2. How do ResNet, ViT, and CLIP respond when spatial structure is altered through translation and patch shuffling? Explain what the results suggest about locality, global organization, and positional sensitivity.
3. To what extent do prediction-level changes agree with changes in feature representations? Discuss at least one informative agreement or mismatch, and compare CLIP’s zero-shot decisions with those of its trained classifier head.
4. Which observed differences can reasonably be attributed to architecture, and which may instead reflect pretraining data, supervision, augmentation, or model capacity? Use your results and failure cases to justify the distinction.

Suggested Repository Structure

Keep transformation generation separate from evaluation so the exact same images can be reused across models.

```
task1/  
  configs/  
  data/  
    make_subset.py  
    make_cue_conflicts.py  
    transforms.py      # color, translation, patch shuffle  
models/  
  backbones.py        # ResNet, ViT, CLIP wrappers  
analysis/  
  evaluate_bias.py  
  feature_similarity.py  
  representation.py  
scripts/run_task1.py  
results/  
README.md
```

2 Unsupervised Domain Adaptation

Introduction and Terminology

In unsupervised domain adaptation (UDA), labeled examples are available from one or more **source domains**, while examples from the **target domain** are available without labels during training. All domains share the same class space, but their visual distributions differ. The objective is to use unlabeled target structure to improve target recognition without weakening the class information learned from the sources.

A **domain gap** is the loss in performance caused by moving from the source distribution to the target distribution. **Domain alignment** encourages source and target features to become less distinguishable. **Marginal alignment** matches the overall feature distributions $P(F(X))$ without explicitly accounting for class, whereas **class-conditional alignment** also uses predicted class structure. **Negative transfer** occurs when adaptation performs worse than the source-only baseline. **Target leakage** occurs whenever target class labels influence training, checkpoint selection, hyperparameters, or method design; target domain identities are permitted, but target class labels are not.

Readings

- **Ganin et al. (2016)** – *Domain-Adversarial Training of Neural Networks*. Focus on the gradient-reversal mechanism and the interaction between classification and domain losses.
- **Long et al. (2018)** – *Conditional Adversarial Domain Adaptation*. Focus on why conditioning the discriminator on class predictions may preserve semantic structure.
- **Optional: Long et al. (2015)** – *Learning Transferable Features with Deep Adaptation Networks*. Useful for understanding kernel-based discrepancy minimization.

- **Optional: Zhao et al. (2019)** – *On Learning Invariant Representations for Domain Adaptation*. Provides a useful warning about treating domain invariance as sufficient for successful adaptation.

Dataset, Models, and Experimental Setup

Use PACS, which contains the same seven object classes in four visual domains: Photo, Art Painting, Cartoon, and Sketch. For both Tasks 2 and 3, use **Sketch as the target domain** and Photo, Art Painting, and Cartoon as the three labeled source domains. In Task 2, all Sketch images may be used without class labels during adaptation. In Task 3, no Sketch image may be used by the training, representation-learning, diagnostic, checkpoint-selection, or hyperparameter-selection procedure. Because Task 2 already exposes the Sketch domain, do not use Task 2 target results to revise any Task 3 method or setting; “unseen target” in Task 3 refers to information available to the Task 3 learning procedure. Reuse the same source splits across both tasks. Under this pairing, the source-only model in this task is also the ERM baseline for Task 3 and should not be retrained differently.

Within each source domain, create a stratified 80/20 training/validation split using seed 6304. Use source training labels to optimize the models and select checkpoints using the mean macro-F1 across the three source validation splits. The complete target domain serves as the unlabeled adaptation set in Task 2 and is evaluated using its labels only after all models, settings, and checkpoints have been fixed. This is a **transductive** UDA protocol: the target images used for final evaluation are visible during adaptation, but their class labels are not.

Use torchvision ResNet-18 with `ResNet18_Weights.IMAGENET1K_V1`. Replace its ImageNet classifier with a seven-class linear classifier head and fine-tune the complete network for every method. Resize images to 256×256 and use a random 224×224 crop with horizontal flipping during training; use a 224×224 center crop for validation and evaluation. Apply the normalization associated with the pretrained weights.

Batch-normalization policy. Because source and target images come from different visual distributions, updating BatchNorm statistics on adaptation batches would make the running mean and variance depend on the source–target mixture and introduce an additional implicit form of adaptation. To keep the comparison focused on the specified alignment objectives, **freeze all BatchNorm running means and variances at their pretrained ImageNet values for every method in Tasks 2 and 3**. The BatchNorm scale and bias parameters (γ and β) remain trainable. In PyTorch, after calling `model.train()`, place only the BatchNorm modules in evaluation mode so their running statistics are not updated; do not place the complete model in evaluation mode.

Train for at most 30 source epochs using AdamW with learning rate 10^{-4} and weight decay 10^{-4} , stopping after five epochs without improvement in mean source-validation macro-F1. Use seed 6304 for every comparison. Each adaptation update should contain eight examples from each source domain and 24 target examples, giving equal total source and target batch sizes; cycle a loader when necessary. Keep initialization, source sampling, augmentation, optimizer, and training budget fixed across methods. Public implementations of individual losses or gradient-reversal layers may be used, but all methods must run through the same training and evaluation pipeline.

Steps

The experiments progress from measuring the unadapted domain gap to testing increasingly structured forms of alignment. The final diagnostic asks whether making domains harder to distinguish actually improves target classification.

1. Source-only ERM

Train the ResNet-18 and classifier head using cross-entropy over the three labeled source domains, with domain-balanced source batches. This model establishes how well ordinary supervised learning transfers without seeing the target domain. Save this checkpoint and reuse it unchanged as the Task 3 ERM baseline. Once the experiment configurations are locked, report its validation performance for each source domain and its final performance on the target.

2. DAN – MMD Alignment

DAN-style alignment adds a discrepancy penalty between source and target representations. Apply Maximum Mean Discrepancy (MMD) to the 512-dimensional feature immediately before the classifier head:

$$\mathcal{L}_{\text{DAN}} = \mathcal{L}_{\text{cls}} + \lambda_{\text{MMD}} \|\mathbb{E}_s[\phi(F(x_s))] - \mathbb{E}_t[\phi(F(x_t))]\|_{\mathcal{H}}^2.$$

Use $\lambda_{\text{MMD}} = 1$ for the main comparison and a sum of three RBF kernels whose bandwidths are 0.5, 1, and 2 times the median pairwise squared feature distance in the current combined batch. The kernel trick evaluates

similarities directly rather than constructing ϕ . This experiment tests whether reducing a statistical source–target discrepancy improves transfer even though the loss does not know which target examples belong to which classes.

3. DANN – Adversarial Alignment

DANN replaces the explicit discrepancy measure with a binary domain discriminator. Attach a discriminator consisting of a 256-unit hidden layer, ReLU, dropout of 0.5, and a two-class output layer to the same 512-dimensional feature. It learns to distinguish pooled source examples from target examples, while a gradient-reversal layer sends the opposite gradient to the backbone. Use the standard schedule

$$\alpha(p) = \frac{2}{1 + \exp(-10p)} - 1,$$

where $p \in [0, 1]$ is training progress, and optimize source classification loss plus domain-classification loss with unit weight. Early in training the representation can learn the class task; later, the stronger reversed gradient increasingly encourages domain confusion. Only source examples contribute to the class loss, while both source and target examples contribute to the domain loss.

4. CDAN – Class-Conditional Adversarial Alignment

Marginal alignment can mix examples from different classes if their domains become similar in the wrong way. CDAN gives the domain discriminator both the feature $f = F(x)$ and the classifier’s probability vector $p = C(f)$ through

$$g(x) = \text{vec}(f \otimes p).$$

Feed $g(x)$ to a discriminator with the same hidden width, activation, dropout, gradient-reversal schedule, and loss weight used for DANN. Do not use entropy conditioning or detach f or p in the required implementation. This comparison tests whether soft class information helps align semantically corresponding regions rather than only the overall domain distributions.

5. Common Evaluation and Alignment Diagnostic

After fixing all checkpoints, report source-validation and target accuracy and macro-F1 for Source-only, DAN, DANN, and CDAN. Also report the target accuracy change relative to Source-only. To measure residual domain information, freeze each backbone and collect equal numbers of source-validation and target features. Using seed 6304, create a 70/30 split and train a balanced logistic-regression classifier with $C = 1$ to distinguish source from target. Its held-out accuracy is the **domain separability** score; 50% indicates chance performance. Compare this score with target recognition rather than assuming that lower separability is automatically better.

Use target labels only at this final analysis stage to compute per-class accuracy. Identify the classes with the largest improvement and degradation relative to Source-only, and inspect their dominant confusions. This establishes whether an aggregate gain hides class-specific negative transfer.

6. Controlled Design Study

Choose one of two bounded studies. Either vary $\lambda_{\text{MMD}} \in \{0.1, 1, 10\}$ for DAN, or vary the maximum gradient-reversal strength in DANN over $\{0.25, 0.5, 1\}$ while retaining the same schedule shape. Keep every other setting fixed. State what you expect the increasing alignment pressure to do to source performance, domain separability, and target recognition before interpreting the results. These target results are for analysis only and may not be used to revise the tested settings.

What to Watch For

- A lower domain-separability score is evidence that domain information is harder to recover, not that class information has been preserved.
- Domain-discriminator accuracy near chance may indicate successful confusion, an undertrained discriminator, or collapsed features. Interpret it alongside classification losses and source performance.
- Pooling three source domains can hide source-specific behavior. Report each source validation domain before presenting the mean.
- A decrease in source performance accompanied by a target gain can be a meaningful trade-off. If both decline, investigate excessive alignment or unstable optimization.
- Do not inspect target-label metrics while choosing checkpoints or settings. Freeze the complete experimental decision before final target evaluation.

Required Evidence

- A table comparing Source-only, DAN, DANN, and CDAN on each source validation domain, mean source accuracy and macro-F1, target accuracy and macro-F1, target accuracy change, and domain separability.
- Classification and alignment or domain-loss curves sufficient to assess whether each adaptation method trained as intended.
- Per-class target accuracy changes and selected confusions or failure cases that support a claim about positive or negative transfer.
- A compact table or plot for the controlled alignment-strength study.

Research Questions

Address the following in your report:

1. How large is the source-to-target domain gap for Source-only ERM, and which classes account for the most important failures?
2. Across the four methods, does lower domain separability correspond to better target recognition? Use aggregate and class-level evidence to identify successful adaptation or negative transfer.
3. How does class-conditional alignment compare with the marginal alignment used by DAN and DANN? Do CDAN's gains or failures support the claim that conditioning helps preserve semantic structure?
4. How does increasing alignment strength change source performance, domain separability, and target performance? What trade-off does your controlled study reveal, and what setting could have been selected without consulting target labels?

Suggested Repository Structure

Use one shared PACS protocol for Tasks 2 and 3, and keep method-specific losses separate from the common training loop.

```
shared/  
  pacs.py  
  pacs_protocol.py  
  splits/pacs_sketch_seed6304.json  
task2/  
  configs/  
    base.yaml  
    source_only.yaml  dan.yaml  dann.yaml  cdan.yaml  
  models/  
    backbone.py  classifier_head.py  domain_discriminator.py  
  methods/  
    source_only.py  dan.py  dann.py  cdan.py  
  evaluation/  
    metrics.py  domain_separability.py  class_analysis.py  
  train.py  
  evaluate_final.py  
  results/  
  README.md
```

3 Domain Generalization

Introduction and Terminology

Domain generalization (DG) asks whether a model trained on several observed domains can generalize to a new domain that was completely unavailable during training. In this task, the labeled source domains expose several forms of appearance variation, but the Sketch domain remains unavailable to the Task 3 learning procedure until final evaluation. You will compare ordinary supervised learning with two different strategies: learning representations that are similar across the observed source domains and seeking parameters whose source loss is locally stable.

Each source domain can be treated as an **environment**. The **risk** $R_e(\theta)$ is the average loss of parameters θ in environment e . **Mean-source performance** averages across source domains, while **worst-source performance** reports the lowest-performing source and reveals whether one environment is being neglected. A **domain-invariant representation** suppresses information that distinguishes domains while retaining information needed for classification. A solution is called **sharp** if a small parameter perturbation causes a large loss increase and **flat** if the loss changes less. Flatness is sensitive to how it is measured, so the diagnostic in this task is evidence about local stability rather than an absolute property of a model.

Readings

- **Long et al. (2015)** – *Learning Transferable Features with Deep Adaptation Networks*. Focus on MMD-based feature alignment; this task will adapt the mechanism to align source domains without using a target domain.
- **Foret et al. (2021)** – *Sharpness-Aware Minimization for Efficiently Improving Generalization*. Focus on the neighborhood objective and two-step optimization procedure.
- **Optional: Gulrajani and Lopez-Paz (2021)** – *In Search of Lost Domain Generalization*. Useful for understanding why strong baselines and source-only model selection are essential in DG comparisons.

Dataset, Models, and Experimental Setup

Reuse the PACS protocol from Task 2: Photo, Art Painting, and Cartoon are the labeled source domains, and Sketch is the unseen target. Use exactly the same source training/validation splits, ResNet-18 initialization, seven-class classifier head, preprocessing, augmentation, domain-balanced sampling, optimizer settings, epoch budget, early stopping rule, and seed 6304. Each training batch should contain eight examples from each source domain. The Source-only model from Task 2 is the ERM baseline here; load its saved checkpoint rather than retraining it under a different configuration.

No Sketch image may be loaded by Task 3 training, source-side diagnostics, checkpoint selection, or hyperparameter selection. In particular, Task 2 results on Sketch must not be used to revise Task 3 architectures, augmentations, losses, or controlled-study settings. Select every checkpoint using mean macro-F1 across the three source validation domains. Load Sketch only in the final Task 3 evaluation script after all Task 3 decisions have been fixed.

The method called **DAN-DG** below is an adaptation of DAN rather than the standard UDA algorithm. Standard DAN aligns labeled source features with features from an unlabeled target. DAN-DG never accesses Sketch; it applies the same MMD mechanism to every pair of observed source domains. Use the same MMD implementation and kernel construction as Task 2 so that the role of target access can later be examined without changing the discrepancy measure.

Steps

The experiments compare three hypotheses. ERM asks whether diverse labeled sources are sufficient, DAN-DG asks whether explicitly removing source-domain differences helps, and SAM asks whether local parameter stability provides a different route to generalization.

1. ERM Baseline

Reuse the Source-only ERM checkpoint trained in Task 2 on the three source domains. Its objective gives every source domain equal influence through domain-balanced batches:

$$\mathcal{L}_{\text{ERM}} = \frac{1}{3} \sum_{e \in \{P, A, C\}} R_e(\theta),$$

where P , A , and C denote Photo, Art Painting, and Cartoon. Report validation accuracy and macro-F1 for each source domain, together with their mean and worst-domain values. These results establish the source-side behavior against which DAN-DG and SAM will be compared before Sketch is evaluated.

2. DAN-DG – Pairwise Source-Domain Alignment

Train a new model using the ERM classification loss plus the average MMD discrepancy over the three unordered

pairs of source domains:

$$\mathcal{L}_{\text{DAN-DG}} = \mathcal{L}_{\text{ERM}} + \frac{\lambda_{\text{DG}}}{3} \sum_{e < e'} \text{MMD}^2(F(X_e), F(X_{e'})).$$

Apply MMD to the 512-dimensional feature before the classifier head. For the main comparison, use $\lambda_{\text{DG}} = 1$ and the same three RBF kernels used in Task 2, with bandwidths 0.5, 1, and 2 times the median pairwise squared distance for each domain pair in the current batch. The classification loss still uses all source labels; MMD only encourages the marginal feature distributions of Photo, Art Painting, and Cartoon to become similar. This tests whether invariance learned entirely from observed domains transfers to Sketch, while retaining the possibility that excessive marginal alignment may mix class structure.

3. SAM – Parameter-Space Stability

Train ERM with standard, non-adaptive Sharpness-Aware Minimization:

$$\min_{\theta} \max_{\|\epsilon\|_2 \leq \rho} \mathcal{L}_{\text{ERM}}(\theta + \epsilon).$$

Use $\rho = 0.05$ for the main comparison and AdamW with the same learning rate and weight decay as ERM. For each source batch, SAM first finds a normalized ascent perturbation and then updates the original parameters using the loss at the perturbed point, requiring two forward/backward passes. Use the shared frozen BatchNorm running-statistics policy from Task 2 during both SAM passes. SAM does not explicitly remove domain information; it tests whether a locally stable source solution transfers better to an unseen visual domain.

4. Common Evaluation and Diagnostics

For ERM, DAN-DG, and SAM, report accuracy and macro-F1 on each source validation domain, their mean and worst-domain values, and final accuracy and macro-F1 on Sketch. Also report the change in Sketch accuracy relative to ERM.

Measure how much source-domain information remains in each representation. Freeze the backbone, collect balanced features from the three source validation sets, and use seed 6304 to create a 70/30 split. Train a multinomial logistic-regression classifier with $C = 1$ to predict Photo, Art Painting, or Cartoon. Its held-out accuracy is the **source-domain separability** score; chance performance is 33.3%. A lower value indicates stronger invariance across the observed sources, but does not by itself establish that class information or unseen-domain performance improved.

Use the same local sharpness proxy for all three models. Select a fixed validation batch containing 32 examples from each source using seed 6304, place the model in evaluation mode, and measure the increase in cross-entropy after one normalized gradient-ascent perturbation with radius 0.05:

$$\Delta_{\text{sharp}} = \mathcal{L}_{\text{val}}(\theta + \epsilon) - \mathcal{L}_{\text{val}}(\theta), \quad \epsilon = 0.05 \frac{\nabla_{\theta} \mathcal{L}_{\text{val}}}{\|\nabla_{\theta} \mathcal{L}_{\text{val}}\|_2}.$$

This is a standardized local diagnostic, not a proof that one model is globally flatter. After configurations are frozen and Sketch labels are available for final analysis, compare per-class accuracy with ERM and inspect the most important improvements, degradations, and confusions.

5. Controlled Design Study

Choose one of two bounded studies. Either vary $\lambda_{\text{DG}} \in \{0.1, 1, 10\}$ for DAN-DG, or vary $\rho \in \{0.01, 0.05, 0.1\}$ for SAM. Keep every other setting fixed and state your expected effect on source performance, the method-specific diagnostic, and Sketch performance before interpreting the results. The main comparison must continue to use $\lambda_{\text{DG}} = 1$ and $\rho = 0.05$; do not use Sketch results to replace these settings with a post-hoc winner.

What to Watch For

- Source-domain invariance is not the same as class invariance. DAN-DG can reduce domain separability while also removing information needed to distinguish classes.
- Strong mean-source performance can hide a weak source domain, while strong worst-source performance need not predict Sketch. Treat both as source-side diagnostics.
- A lower sharpness proxy supports a claim about local stability under the specified perturbation; it does not establish that the entire loss landscape is flatter or that domain shifts have been solved.

- DAN in Task 2 and DAN-DG in Task 3 use the same discrepancy mechanism but different information. The former sees unlabeled Sketch; the latter aligns only observed sources.
- Do not inspect Sketch examples or labels to explain a training choice. Target examples may be examined only as part of the final failure analysis.

Required Evidence

- A table comparing ERM, DAN-DG, and SAM on each source validation domain, mean-source, worst-source, and Sketch accuracy and macro-F1, together with the change in Sketch accuracy relative to ERM.
- Source-domain separability and the common sharpness proxy for all three models.
- Training curves that include classification loss and, where applicable, the MMD penalty.
- A compact table or plot for the controlled DAN-DG or SAM study.
- Per-class Sketch changes and selected failures that support the conclusions, including a comparison with the corresponding Task 2 results.

Research Questions

Address the following in your report:

1. How well do mean-source and worst-source validation performance predict performance on Sketch? Which class-level failures are not visible from the aggregate source results?
2. Does DAN-DG make the observed source domains less distinguishable, and does that invariance improve Sketch recognition? Identify any evidence that source alignment instead removed class-discriminative information.
3. Does SAM reduce the common sharpness proxy relative to ERM and DAN-DG? Explain whether the ranking by local stability agrees with the ranking by Sketch performance.
4. Relative to the shared ERM baseline, what does the comparison between target-aware DAN in Task 2 and target-free DAN-DG in Task 3 suggest about the value of unlabeled Sketch data? What limitations prevent this from being a complete causal attribution?

Suggested Repository Structure

Reuse the shared PACS data and split code from Task 2. Keep source-only model selection separate from the script that loads Sketch for final evaluation.

```
task3/
  configs/
    erm.yaml  dan_dg.yaml  sam.yaml
  models/
    backbone.py  classifier_head.py
  methods/
    erm.py  dan_dg.py  sam.py
  selection/
    source_validation.py
  evaluation/
    domain_metrics.py
    source_domain_separability.py
    sharpness.py
  train.py
  evaluate_sketch.py
  results/
  README.md
```

4 Open-Set Recognition (OSR)

Introduction and Terminology

A closed-set classifier assumes that every test input belongs to one of its training classes. When this assumption fails, the classifier must still choose a known label and may do so with high confidence. Open-set recognition (OSR) augments classification with the ability to reject inputs that do not appear to belong to any known class.

In this task, CIFAR-10 provides the ten **known classes**, while selected CIFAR-100 classes appear only during final evaluation as **unknowns**. A **near-semantic unknown** is visually or semantically similar to a known class; a **far-semantic unknown** has less overlap with the known label space. **Closed-set accuracy (CSA)** measures ordinary classification on known examples before rejection. An **unknownness score** assigns larger values to inputs that appear less compatible with the known classes, and a **rejection threshold** converts that score into a known/unknown decision. A **post-hoc score** uses a fixed trained model, whereas **placeholder learning** changes the classifier and representation using proxy unknown features constructed only from known training data.

Readings

- **Vaze et al. (2022)** – *Open-Set Recognition: a Good Closed-Set Classifier is All You Need?* Focus on the relationship between closed-set classifier quality and Maximum Logit Score.
- **Zhou et al. (2021)** – *Learning Placeholders for Open-Set Recognition*. Focus on classifier placeholders and the use of manifold mixup to construct data placeholders between different known classes.
- **Optional: Hendrycks and Gimpel (2017)** – *A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks*. Provides the motivation for maximum-softmax scoring.
- **Optional: Liu et al. (2020)** – *Energy-based Out-of-distribution Detection*. Useful for understanding energy as a function of all classifier logits.
- **Optional: Chen et al. (2020)** – *Learning Open Set Network with Discriminative Reciprocal Points*. Read this if you choose the optional RPL extension; focus on reciprocal points as representations of extra-class space and the open-space regularization objective.

Dataset, Models, and Experimental Setup

Use all ten CIFAR-10 classes as known. Create a stratified 90/10 split of the official training partition using seed 6304, use only the training portion for optimization, and select checkpoints by CIFAR-10 validation accuracy. Use the complete CIFAR-10 test set for final known-class evaluation.

Unknowns come from the following fixed CIFAR-100 test classes:

Group	CIFAR-100 fine classes
Near unknown	bus, pickup_truck, motorcycle, tractor, wolf, fox, leopard, camel
Far unknown	bottle, bowl, chair, clock, keyboard, mushroom, sunflower, wardrobe

Each group contains 800 images. The grouping is fixed and may not be revised after seeing results. CIFAR-100 training images may not be used for training, checkpoint selection, score design, or threshold selection.

Use the same CIFAR-appropriate ResNet-18 architecture throughout Task 4. Train Vanilla and GCSC from random initialization. PROSER is initialized from the selected Vanilla checkpoint as specified below. Replace the ImageNet 7×7 , stride-2 first convolution with a 3×3 , stride-1 convolution and remove the initial max-pooling layer. Operate on the original 32×32 images. Unless a step explicitly changes the classifier output or augmentation, use:

- random crop to 32×32 with 4-pixel padding and random horizontal flipping;
- SGD with learning rate 0.1, momentum 0.9, weight decay 5×10^{-4} , and cosine decay;
- batch size 128, 100 epochs, and seed 6304.

Save the checkpoint with the highest CIFAR-10 validation accuracy. Unknown examples may be evaluated only after every checkpoint and score definition has been fixed.

Steps

Begin with one ordinary classifier and compare several scores on its fixed outputs. Then compare stronger positive augmentation with PROSER, which learns classifier and data placeholders for unknown inputs.

1. Vanilla Closed-Set Baseline

Train the ten-class ResNet-18 with cross-entropy. Freeze the selected checkpoint and extract its penultimate feature $f(x)$ and logits $z(x) = (z_1, \dots, z_{10})$ for the CIFAR-10 training, validation, and test examples and the fixed CIFAR-100 evaluation examples. Report CIFAR-10 test accuracy. After training is complete, inspect unknown inputs that receive confident known-class predictions as part of the final failure analysis. These observations are for analysis only and must not be used to revise the model, score definition, hyperparameters, or rejection threshold.

2. Post-hoc Novelty Scores

Keep the vanilla model fixed and express every method as an unknownness score $u(x)$, where larger values indicate greater novelty. With $p_k(x) = \exp(z_k) / \sum_j \exp(z_j)$, implement:

$$\begin{aligned} u_{\text{MSP}}(x) &= 1 - \max_k p_k(x), & u_{\text{MLS}}(x) &= -\max_k z_k(x), \\ u_{\text{Energy}}(x) &= -\log \sum_k \exp(z_k(x)), & u_{\text{Mah}}(x) &= \min_c (f(x) - \mu_c)^\top \Sigma^{-1} (f(x) - \mu_c). \end{aligned}$$

MSP measures normalized confidence, MLS retains absolute logit magnitude, Energy uses all logits, and Mahalanobis measures distance from known-class feature clusters. For Mahalanobis, estimate class means μ_c and one shared diagonal covariance Σ from unaugmented CIFAR-10 training features, adding 10^{-6} to every diagonal entry. All four scores must use exactly the same saved logits and features.

3. GCSC – Strong Closed-Set Classifier + MLS

Train a second ten-class ResNet-18 using exactly the vanilla recipe with one change: insert `RandAugment(num_ops=2, magnitude=9)` after the crop and flip and before conversion and normalization. Keep the initialization procedure, optimizer, schedule, batch size, epochs, seed, and checkpoint rule unchanged. Evaluate it with MLS. This controlled comparison tests whether an augmentation-induced change in closed-set accuracy is accompanied by better rejection, particularly for near unknowns.

4. PROSER – Classifier and Data Placeholders

PROSER prepares a closed-set classifier for unknown inputs by learning two types of **placeholders**. The key idea is to create training signals for “unknown-like” regions without using any real unknown CIFAR-100 examples.

Initialize PROSER from the selected Vanilla checkpoint and append five randomly initialized **dummy classifiers**. Fine-tune the complete model for 50 epochs using SGD with learning rate 10^{-3} , momentum 0.9, weight decay 5×10^{-4} , cosine decay, batch size 128, and seed 6304.

PROSER uses two complementary training objectives:

- **Classifier placeholders.** For an ordinary CIFAR-10 example, the correct known-class response should remain the largest. However, once that correct class is excluded, PROSER encourages one of the dummy classifiers to become the strongest remaining response. Intuitively, the dummy classifiers learn alternative regions around the known classes that can later provide evidence for rejection.
Use the classifier-placeholder loss defined in [Zhou et al. \(2021\)](#). Use $\beta = 1$ for the classifier-placeholder loss.
- **Data placeholders.** Since no true unknown examples are available during training, PROSER creates synthetic unknown-like representations using *manifold mixup*. Mix representations belonging to two different CIFAR-10 classes so that the resulting feature lies between their learned class regions. These mixed representations are then trained toward the dummy classifiers rather than either original known class.

Split each mini-batch into two equal parts: use the first half for classifier-placeholder training and the second half for manifold-mixup data placeholders.

For the required CIFAR ResNet-18, perform manifold mixup **after layer2 and before layer3**. For two examples from different classes, let

$$h_i = \phi_{\text{pre}}(x_i), \quad h_j = \phi_{\text{pre}}(x_j),$$

where ϕ_{pre} denotes the network up to **layer2**. Sample

$$\lambda \sim \text{Beta}(2, 2)$$

and construct

$$\tilde{h} = \lambda h_i + (1 - \lambda) h_j, \quad y_i \neq y_j.$$

Pass $\tilde{h}$ through the remainder of the network and train the resulting example toward the dummy classifiers.

These mixed representations act as **proxy unknowns**: they often lie between known-class regions and therefore provide examples of locations where the classifier should avoid making a confident known-class prediction. This can encourage more compact known-class regions and improve unknown rejection.

Use the data-placeholder loss from the PROSER paper with $\gamma = 0.1$. The required training objective therefore follows the paper’s combination of the classifier-placeholder and data-placeholder losses. Refer to the paper for the exact loss construction and handling of the dummy classifiers.

No CIFAR-100 image may participate in either placeholder construction. Select the checkpoint using CIFAR-10 validation accuracy only.

For evaluation, report PROSER with MLS on the ten known-class logits so it can be compared directly with Vanilla and GCSC. In addition, evaluate PROSER using the placeholder-based detection score from the reference implementation. Follow the paper/reference code for combining the strongest dummy response with the known-class responses and calibrate the rejection rule using CIFAR-10 validation data only.

5. Optional Extension – Reciprocal Point Learning (RPL)

As an optional extension, implement Reciprocal Point Learning (RPL) following [Chen et al. \(2020\)](#).

Most classifiers learn what each class *is*. RPL instead learns what each class *is not* using a learned **reciprocal point**. A feature from class k is pushed away from the reciprocal point for class k , so greater distance gives stronger evidence for that class. At test time, examples that are not sufficiently far from any reciprocal point can be rejected as unknown. RPL also uses an open-space regularization term to keep the learned feature space bounded.

Use the same CIFAR-appropriate ResNet-18 and train using CIFAR-10 only. No CIFAR-100 image may influence training, checkpoint selection, or hyperparameter selection. You may adapt the authors’ or another public implementation, but clearly identify:

- how the reciprocal points are represented and learned;
- how distance to a reciprocal point determines the known-class score;
- how the open-space regularization term constrains the feature space;
- which RPL score is used to reject unknown examples at test time.

Evaluate RPL using the same near/far CIFAR-100 split, AUROC, and validation-calibrated rejection protocol as the required methods. If completed, add RPL as an additional row in the trained-model comparison and discuss whether learning what each class *is not* improves rejection relative to Vanilla, GCSC, and PROSER.

6. Common Evaluation and Failure Analysis

Evaluate every score separately on Known vs. Near, Known vs. Far, and Known vs. All Unknowns. Report AUROC for all three comparisons. For each model and score, choose a threshold equal to the 95th percentile of unknownness on the CIFAR-10 validation set; accept x when $u(x) \leq \tau$. This uses known validation data only and aims to accept 95% of known examples. Report the achieved CIFAR-10 test acceptance rate and the near- and far-unknown rejection rates. Equivalently, report FPR@95TPR as the fraction of unknown examples incorrectly accepted under this convention.

First compare MSP, MLS, Energy, and Mahalanobis on the same vanilla model. Then compare Vanilla, GCSC, and PROSER using MLS; include a second row for PROSER using its placeholder-based detection score. For PROSER, compute CSA using only the ten known-class logits so that known-class classification and rejection remain distinct.

Plot score distributions or ROC curves for MSP, MLS, and Mahalanobis on one compact multi-panel figure. Using the vanilla MLS threshold, inspect at least three incorrectly accepted near unknowns and three incorrectly accepted far unknowns. Record the unknown class, predicted CIFAR-10 class, score, and threshold, and distinguish semantically plausible confusions from surprising failures.

What to Watch For

- Near and far unknowns represent different semantic difficulty. Rejecting a **wardrobe** does not imply that an unseen **wolf** can be distinguished from **dog**.
- CIFAR-100 is evaluation-only. Unknown examples may not influence training, checkpoint selection, score definitions, or thresholds.
- Confidence, logit magnitude, aggregate logit evidence, and feature distance are different signals. Examine where their rankings agree and disagree.
- Better closed-set accuracy need not improve rejection. Compare changes in CSA with near and far OSR rather than assuming they move together.
- Feature interpolations are proxy unknowns, not real unknown classes. They may tighten the boundaries between known classes without representing every direction from which an unknown can arrive.
- AUROC measures ranking over all thresholds; validation-calibrated rejection measures behavior at one operating point. Report and interpret both.

Required Evidence

- One table comparing MSP, MLS, Energy, and Mahalanobis on the frozen vanilla model using near, far, and all-unknown AUROC and validation-calibrated rejection.
- One table comparing Vanilla, GCSC, and PROSER using CSA and near/far OSR metrics, with MLS as the common score and the PROSER placeholder-based detection score as an additional row.
- One compact score-distribution or ROC figure for the three specified scores.
- At least three informative near-unknown failures and three far-unknown failures, including the unknown class, predicted known class, score, and threshold.
- A concise analysis of positive augmentation versus PROSER, including how classifier and data placeholders affect the trade-off between known-class recognition and unknown rejection.

Research Questions

Address the following in your report:

1. How does semantic similarity affect rejection? Which near and far classes are most often accepted, which CIFAR-10 labels absorb them, and which failures are semantically plausible?
2. What do MSP, MLS, Energy, and Mahalanobis each capture? Use agreements and disagreements among their scores to explain when confidence, logit magnitude, or feature distance is most informative.
3. Do GCSC's random augmentations improve closed-set accuracy, open-set recognition, both, or neither? Explain why its effect differs or remains similar for near and far unknowns.
4. Does PROSER improve rejection beyond the Vanilla and GCSC models, and what CSA trade-off results? Are its gains consistent with classifier and data placeholders tightening the boundaries between known classes, and where does interpolation between known classes remain insufficient?

Suggested Repository Structure

Keep dataset construction, model training, score extraction, and evaluation separate so that every score receives identical examples.

```
task4/  
  configs/  
    vanilla.yaml  
    gcsc.yaml  
    proser.yaml  
    rpl.yaml          # optional
```

```

methods/
  vanilla.py
  gcsc.py
  proser.py
  manifold_mixup.py
  rpl.py          # optional

data/
  cifar10.py
  cifar100_unknowns.py
  make_splits.py

models/
  resnet_cifar.py

scores/
  msp.py  mls.py  energy.py  mahalanobis.py

cache/
  # saved features/logits only

evaluation/
  metrics.py
  thresholds.py
  failure_analysis.py

train.py
extract_outputs.py
evaluate_osr.py
results/
README.md

```

5 Synthesis of Results & Report Writing Guidelines:

After completing the four tasks, synthesize the experiments into one coherent study rather than four disconnected homework answers. The unifying question is: *what should a robust representation preserve, what should it become invariant to, and when should it reject an input rather than force a prediction?*

Cross-task Analysis: Use the individual task results to develop a broader argument about what a useful representation should preserve, suppress, and reject. In particular, discuss:

- **Visual cues and distribution shift.** Relate the shape, texture, color, and spatial sensitivities observed in Task 1 to the failure patterns observed under the PACS domain shifts in Tasks 2 and 3. Do the results suggest a consistent vulnerability to particular kinds of visual change? Treat this as a qualitative comparison rather than a causal claim, since the tasks use different datasets, architectures, and training procedures.
- **Invariance versus discriminability.** Across Tasks 2 and 3, examine whether making domains less distinguishable actually improves recognition. Compare domain separability, source performance, class-level failures, and target performance to identify cases where invariance removes noise and cases where it also removes class-discriminative information.
- **The importance of target domain access.** Use the shared ERM baseline and the comparison between target-aware DAN in Task 2 and target-free DAN-DG in Task 3 to discuss what unlabeled target data contributes. Does access to Sketch allow more useful alignment, and what limitations prevent the comparison from establishing that target access alone caused the difference?
- **Recognition versus rejection.** Connect Task 4 to the earlier tasks by asking whether representations that improve ordinary classification also become safer under semantic novelty. Compare Vanilla, GCSC, and PROSER and distinguish improvements in closed-set accuracy from improvements in unknown rejection. Use the near-versus-far unknown results to identify where a confident classifier should instead abstain.
- **Overall synthesis.** Based on evidence across all four tasks, identify which information a robust representation should preserve for classification, which variations it should become invariant to, and which inputs should be treated as outside the learned label space rather than forced into a known class.

Deliverables:

- **GitHub Repository:** code, configuration files, split logic/indices, environment specification, machine-readable results, and a top-level README explaining how to reproduce each task. Do not commit raw datasets or unnecessary large checkpoints.
- **PDF Report:** 8 pages of main content in the course NeurIPS format introduced in PA0. References do not count toward the limit. A short report appendix may contain implementation details or extra results, but graders are not required to read it and it cannot replace required evidence.
- **LMS Submission:** upload the PDF and submit an accessible GitHub link in the format announced on LMS.

Suggested Report Organization: Abstract; Introduction and research questions; Experimental Setup/Methods; Results for Tasks 1–4; Cross-task Discussion; Conclusion; References. Keep paper-method summaries concise in the report and spend your limited space on controlled evidence, comparisons, failure analysis, and interpretation.

Before You Submit: Check that no target labels influenced Task 2 model selection, no Sketch data were consumed by Task 3 training or model selection, and Task 2 target results did not influence Task 3 settings, and no real unknown examples influenced Task 4 training or threshold selection. Every reported number should trace to a saved result file or reproducible command, and every external implementation should be attributed.

Appendix: Git and Version Control for Research Code

This appendix is a practical add-on. The goal is to make your repository behave like a small research codebase: experiments should be recoverable, changes should be attributable, and the exact code used for the report should be identifiable.

Recommended Overall Repository Layout:

```
pa1-beyond-iid/
  README.md
  requirements.txt          # or environment.yml / pyproject.toml
  .gitignore
  common/                  # only genuinely shared utilities
    seed.py logging.py plotting.py metrics.py
  task1/
  task2/
  task3/
  task4/
  report/
  figures/
```

Keep the task boundaries obvious. Move a helper into `common/` only when multiple tasks genuinely share it. Keep raw datasets outside Git; your README should document how to download/prepare them.

1. Create or clone the repository:

```
git clone <repository-url>
cd <repository>
# or for an existing local folder:
git init
git remote add origin <repository-url>
```

2. Add a `.gitignore` early: Ignore datasets, environments, caches, and large checkpoints, but keep small CSV/JSON result files that support the report.

```
data/
.venv/
__pycache__/
*.pyc
wandb/
checkpoints/
*.pth
*.pt
```

3. Commit coherent changes regularly:

```
git status
git add task2/methods/dann.py task2/configs/dann.yaml
git commit -m "Implement DANN gradient reversal baseline"
git push
```

A useful commit should capture one coherent change: a new baseline, a bug fix, a new metric, or a reproducibility improvement.

Avoid making your first meaningful commit at the deadline.

4. Use branches for risky experiments:

```
git switch -c task4-rpl
# implement and test
git add .
git commit -m "Add reciprocal point training objective"
git switch main
git merge task4-rpl
```

After a merge, rerun at least a smoke test before trusting previously working experiments.

5. Useful inspection and recovery commands:

```
git log --oneline --graph --decorate -15
git diff
git diff --staged
git restore path/to/file.py
git show <commit>:task3/methods/sam.py
```

Do not run destructive Git commands you do not understand. If you accidentally add a large dataset/checkpoint, stop tracking it and add an appropriate pattern to `.gitignore` before pushing more copies.

6. Freeze the submitted code state: When the final results and report are fixed, commit and push the code.

```
git add .
git commit -m <meaningful commit message>
git push origin main
```